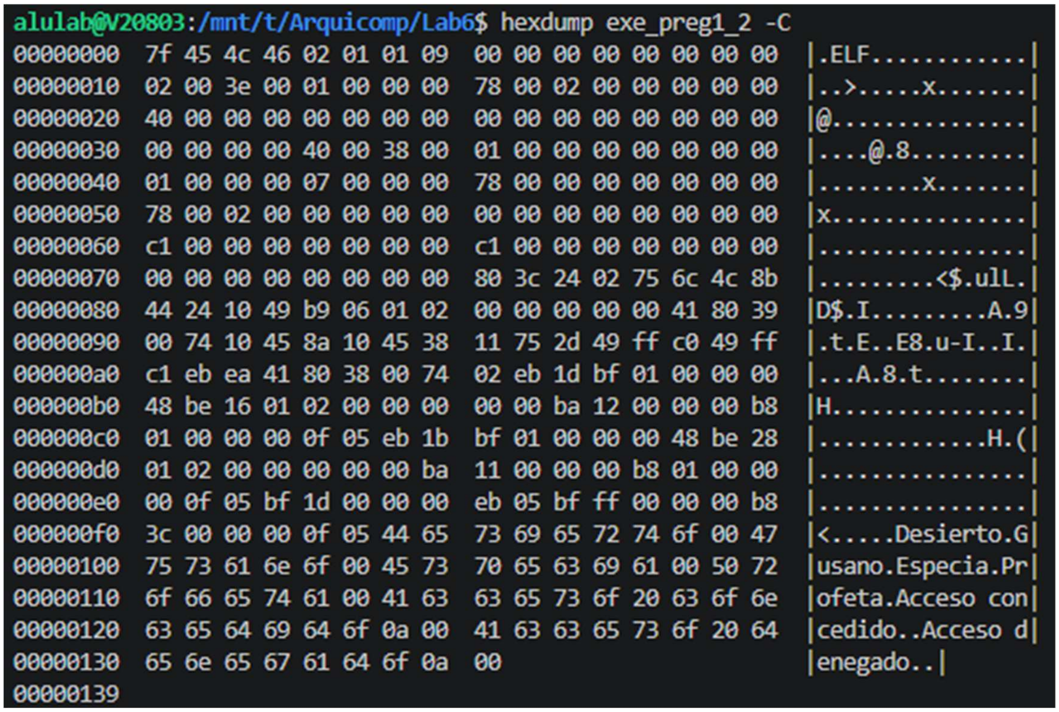
Código de estudiante: 20240282

Pregunta 1:

La contraseña es: “Especia”
Se obtuvo realizando el siguiente procedimiento:

1. hexdump exe_preg1_2 -C



```
alulab@V20803:/mnt/t/Arquicomp/Lab6$ hexdump exe_preg1_2 -C
00000000  7f 45 4c 46 02 01 01 09  00 00 00 00 00 00 00 00  |.ELF............|
00000010  02 00 3e 00 01 00 00 00  78 00 02 00 00 00 00 00  |..>.....x.......|
00000020  40 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |@...............|
00000030  00 00 00 00 40 00 38 00  01 00 00 00 00 00 00 00  |....@.8.........|
00000040  01 00 00 00 07 00 00 00  78 00 00 00 00 00 00 00  |........x.......|
00000050  78 00 02 00 00 00 00 00  00 00 00 00 00 00 00 00  |x...............|
00000060  c1 00 00 00 00 00 00 00  c1 00 00 00 00 00 00 00  |................|
00000070  00 00 00 00 00 00 00 00  80 3c 24 02 75 6c 4c 8b  |.........<$.ulL.|
00000080  44 24 10 49 b9 06 01 02  00 00 00 00 00 41 80 39  |D$.I.........A.9|
00000090  00 74 10 45 8a 10 45 38  11 75 2d 49 ff c0 49 ff  |.t.E..E8.u-I..I.|
000000a0  c1 eb ea 41 80 38 00 74  02 eb 1d bf 01 00 00 00  |...A.8.t........|
000000b0  48 be 16 01 02 00 00 00  00 00 ba 12 00 00 00 b8  |H...............|
000000c0  01 00 00 00 0f 05 eb 1b  bf 01 00 00 00 48 be 28  |.............H.(|
000000d0  01 02 00 00 00 00 00 ba  11 00 00 00 b8 01 00 00  |................|
000000e0  00 0f 05 bf 1d 00 00 00  eb 05 bf ff 00 00 00 b8  |................|
000000f0  3c 00 00 00 0f 05 44 65  73 69 65 72 74 6f 00 47  |<.....Desierto.G|
00000100  75 73 61 6e 6f 00 45 73  70 65 63 69 61 00 50 72  |usano.Especia.Pr|
00000110  6f 66 65 74 61 00 41 63  63 65 73 6f 20 63 6f 6e  |ofeta.Acceso con|
00000120  63 65 64 69 64 6f 0a 00  41 63 63 65 73 6f 20 64  |cedido..Acceso d|
00000130  65 6e 65 67 61 64 6f 0a  00                       |enegado..|
00000139
```

2. Probamos palabra por palabra hasta dar con la correcta

Ahora para el calculo de la cantidad de bytes de instrucciones

Ocupan 67 bytes = 0x43 bytes

El programa en total ocupa 0x139

Los headers ocupan 120 bytes = 0x78

Bytes Instrucciones = Bytes Totales – (Bytes Headers + Bytes Cadenas Caracteres)

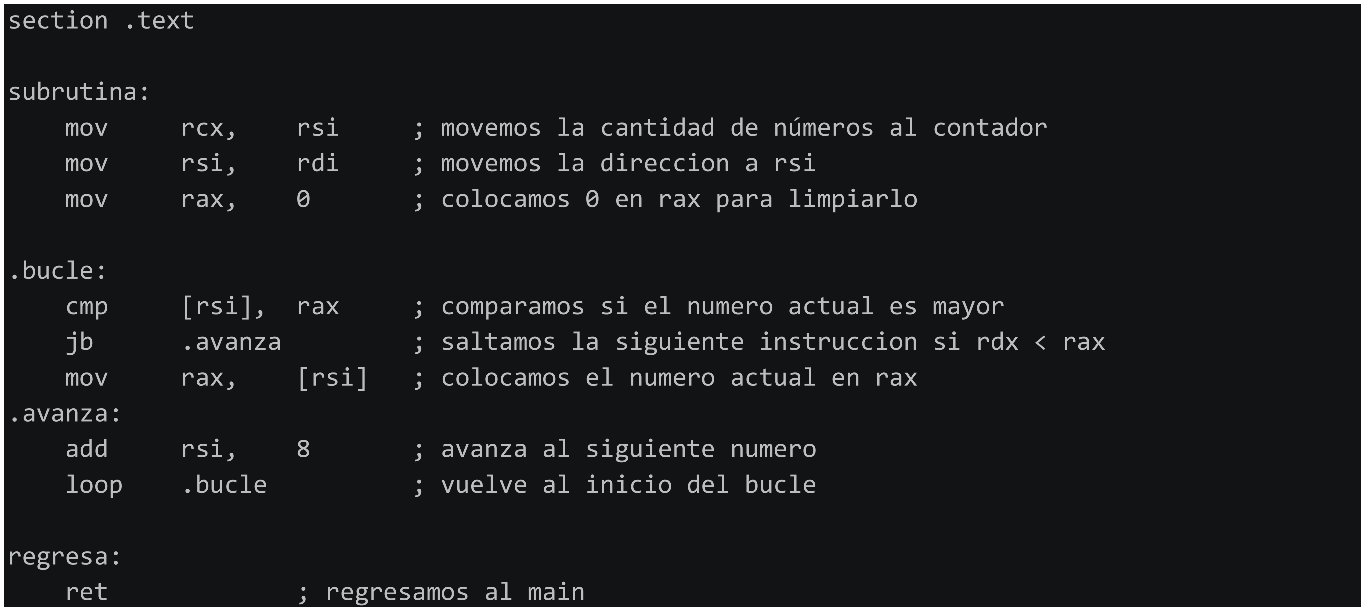
Bytes Instrucciones = 0x139 – (0x43 + 0x78)

Bytes Instrucciones = 0x139 – 0xBB

Bytes Instrucciones = 0x7E = 126 bytes

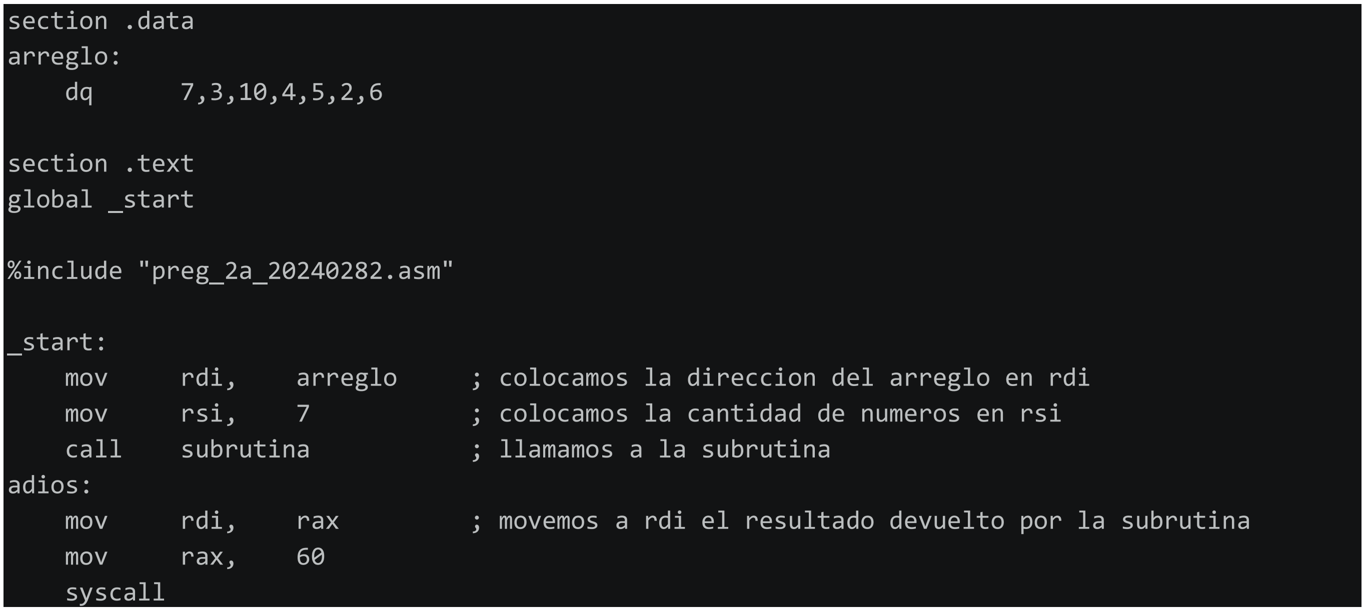
Pregunta 2:

a).- Presente aquí el código de la subrutina.



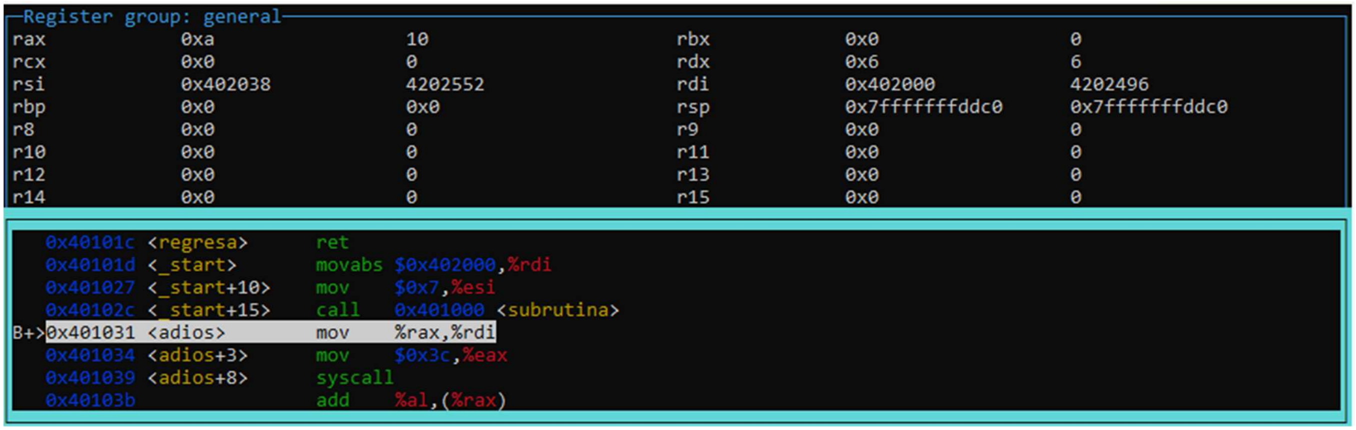
```
section .text

subrutina:
    mov     rcx,    rsi     ; movemos la cantidad de números al contador
    mov     rsi,    rdi     ; movemos la direccion a rsi
    mov     rax,    0       ; colocamos 0 en rax para limpiarlo

.bucle:
    cmp     [rsi],  rax     ; comparamos si el numero actual es mayor
    jb      .avanza         ; saltamos la siguiente instruccion si rdx < rax
    mov     rax,    [rsi]   ; colocamos el numero actual en rax
.avanza:
    add     rsi,    8       ; avanza al siguiente numero
    loop    .bucle          ; vuelve al inicio del bucle

regresa:
    ret             ; regresamos al main
```

b).- Presente aquí el código principal en sintaxis estándar.



```
section .data
arreglo:
    dq      7,3,10,4,5,2,6

section .text
global _start

%include "preg_2a_20240282.asm"

_start:
    mov     rdi,    arreglo     ; colocamos la direccion del arreglo en rdi
    mov     rsi,    7           ; colocamos la cantidad de numeros en rsi
    call    subrutina           ; llamamos a la subrutina
adios:
    mov     rdi,    rax         ; movemos a rdi el resultado devuelto por la subrutina
    mov     rax,    60
    syscall
```

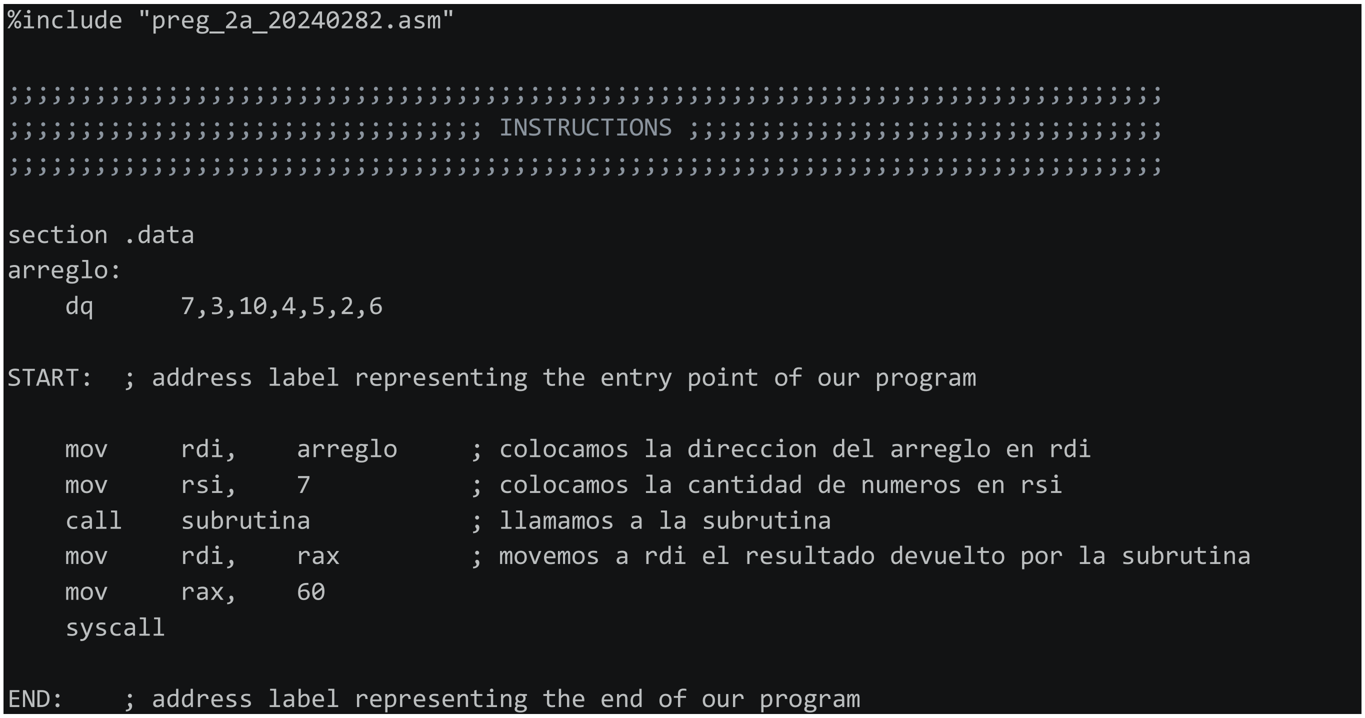
c).- Presente aquí la captura de pantalla solicitada.



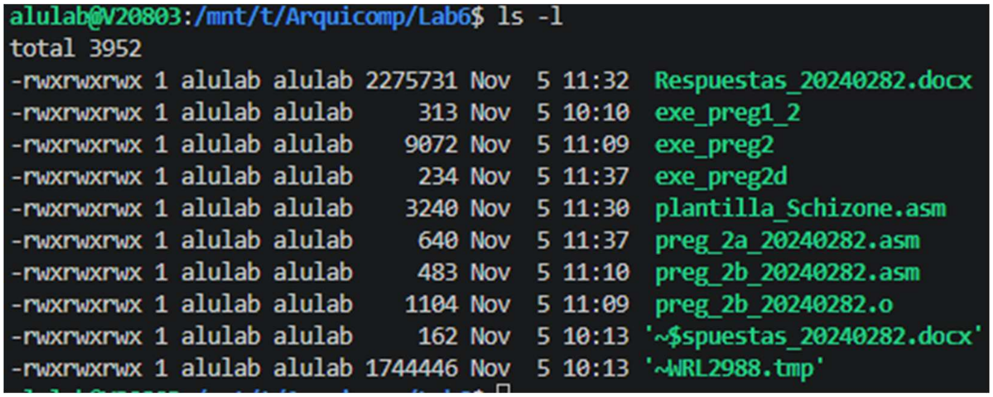
```
Register group: general
rax            0xa                 10                  rbx            0x0                 0
rcx            0x0                 0                   rdx            0x6                 6
rsi            0x402038            4202552             rdi            0x402000            4202496
rbp            0x0                 0x0                 rsp            0x7fffffffddc0      0x7fffffffddc0
r8             0x0                 0                   r9             0x0                 0
r10            0x0                 0                   r11            0x0                 0
r12            0x0                 0                   r13            0x0                 0
r14            0x0                 0                   r15            0x0                 0

    0x40101c <regresa>       ret
    0x40101d <_start>        movabs $0x402000,%rdi
    0x401027 <_start+10>     mov    $0x7,%esi
    0x40102c <_start+15>     call   0x401000 <subrutina>
B+> 0x401031 <adios>         mov    %rax,%rdi
    0x401034 <adios+3>       mov    $0x3c,%eax
    0x401039 <adios+8>       syscall
    0x40103b                 add    %al,(%rax)
```

Los números utilizados para la ejecución de este programa fueron: 7, 3, 10, 4, 5, 2 y 6

d).- Presente aquí el código principal usando la plantilla de Schizone.



```
%include "preg_2a_20240282.asm"

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;; INSTRUCTIONS ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

section .data
arreglo:
    dq      7,3,10,4,5,2,6

START:  ; address label representing the entry point of our program

    mov     rdi,    arreglo     ; colocamos la direccion del arreglo en rdi
    mov     rsi,    7           ; colocamos la cantidad de numeros en rsi
    call    subrutina           ; llamamos a la subrutina
    mov     rdi,    rax         ; movemos a rdi el resultado devuelto por la subrutina
    mov     rax,    60
    syscall

END:    ; address label representing the end of our program
```



```
alulab@V20803:/mnt/t/Arquicomp/Lab6$ ls -l
total 3952
-rwxrwxrwx 1 alulab alulab 2275731 Nov  5 11:32  Respuestas_20240282.docx
-rwxrwxrwx 1 alulab alulab     313 Nov  5 10:10  exe_preg1_2
-rwxrwxrwx 1 alulab alulab    9072 Nov  5 11:09  exe_preg2
-rwxrwxrwx 1 alulab alulab     234 Nov  5 11:37  exe_preg2d
-rwxrwxrwx 1 alulab alulab    3240 Nov  5 11:30  plantilla_Schizone.asm
-rwxrwxrwx 1 alulab alulab     640 Nov  5 11:37  preg_2a_20240282.asm
-rwxrwxrwx 1 alulab alulab     483 Nov  5 11:10  preg_2b_20240282.asm
-rwxrwxrwx 1 alulab alulab    1104 Nov  5 11:09  preg_2b_20240282.o
-rwxrwxrwx 1 alulab alulab     162 Nov  5 10:13 '~$spuestas_20240282.docx'
-rwxrwxrwx 1 alulab alulab 1744446 Nov  5 10:13 '~WRL2988.tmp'
```

El archivo ejectuable generado con la plntilla de Schizone es “exe_preg2d”, pesa 234 bytes y quitándole lo que ocupan los bytes de los números (56 bytes) tiene un peso de 178 bytes

e) Presente aquí sus explicaciones.

El programa principal primero define el arreglo de números con un dq ya que son números de 64 bits. Luego lo que hace es cargar la información necesaria para que la subrutina pueda funcionar, RDI como primer argumento, en el cual se coloca la dirección del arreglo, como segundo argumento usa RSI, donde se guarda la cantidad de números que hay en el arreglo. Llama a la subrutina y al volver copia el valor de RAX en RDI y finaliza el programa.
La subrutina copia RSI a RCX para utilizar la cantidad de números como contador, también copia RDI a RSI para manejar el arreglo y limpia RAX colocándolo en 0. Luego entra en un bucle, donde compara el valor actual del arreglo con RAX, y si es mayor a el entonces lo coloca como nuevo valor de RAX, luego agrega 8 a RSI para pasar al siguiente numero y vuelve al inicio del bucle, esto se hace tantas veces como números haya en el arreglo. Finalmente regresa al main con RAX como el mayor valor que hay en el arreglo de números.
RDI fue usado como primer argumento para pasarle a la subrutina la dirección del arreglo.
RSI fue usado como segundo argumento para pasarle a la subrutina la cantidad de números del arreglo y la subrutina lo uso como puntero que apunta a uno de los números.
RAX fue usado para devolver el valor que se obtuvo en la subrutina al main.

Pregunta 3:

a).- Presente aquí el código de la subrutina.

b).- Presente aquí el código principal en sintaxis estándar.

c).- Presente aquí la captura de pantalla solicitada.

d).- Presente aquí el código principal usando la plantilla de Schizone.

e) Presente aquí sus explicaciones.